CpnetSocketServer
July 3, 2026

Table of Contents

Introduction..... 1
Native Files..... 1
Server Configuration..... 2
Starting the Server..... 3
Diablo 630 Implementation..... 4
Network Boot..... 5
Network Boot Protocols..... 5
Network Boot “SYS” Image Format..... 6
Network Boot “SYS” File Selection..... 7
Network Loader Image Format..... 8
Network Loader File Selection..... 8

Introduction

CpnetSocketServer is a JAVA program the implements a CP/NET server, with CP/M 3 extensions. It listens for socket connection requests and provides CP/M-style access to native file folders on the host.

Multiple servers may be running on the same host, provided they don’t conflict. They must have different root_dir, port, and sid values at a minimum.

Native Files

Native files must have lower-case only names. Mixed-case filenames will cause unpredictable results. All files created by CP/M will be in lower-case.

The file’s write permission is used to reflect the CP/M RO attribute. CP/M programs that change a file’s RO attribute will change the native file’s write permission.

The file’s execute permission is used to reflect the CP/M SYS attribute. Note that Windows will always show files as executable, and thus files on a Windows host will always have the SYS attribute set. Also remember that CP/M normally hides files that have the SYS attribute set. There is a server configuration setting the disables the SYS attribute, to avoid these issues on Windows hosts.

The CP/M ARCHIVE attribute is not supported.

Files that are not an even multiple of 128 bytes in size will be padded to a 128-byte multiple, using Ctrl-Z (EOF, 0x1a), when reading. Writing to a file always involves a full 128-byte record, so no additional padding is performed. The CP/M 3 feature “Set File Byte Count” will truncate a file to a specific, arbitrary, number of bytes, after which the file may no longer be an even multiple of 128 bytes.

Server Configuration

The server is configured using a “configuration file”, which is plan text formatted as “property = value” lines. The configuration file to be used is specified on the commandline using the parameter “**conf=***file*”. The environment variable **CPNET_CONFIG** may also specify the configuration file. If nothing is specified, the server will look in the current directory for “**cpnetrc**” and then the user’s home directory for “**.cpnetrc**”. Comments in the file start with ‘#’ as the first non-space character of the line. Property values may use backslash (‘\’) to extend long values to the following line.

Many properties may be specified on the commandline, using a “parameter=value” format. The parameter names are the property names with the “cpnetserver_” prefix removed.

The commandline parameter “wait”, which does not have a property equivalent, will cause the server to wait for the network interface to come up. This may be used when starting the server at boot time, for example when using the cron target “@reboot”. Note, however, that this option causes the server to wait forever, if the network interface is no longer functioning.

The following properties are recognized:

cpnetserver_log = *log-file*
: Diverts stderr to *log-file.*

cpnetserver_log_time
: Prepends timestamp to stderr (log file) output.

cpnetserver_host = *host-or-ip*
: Specifies the host network address to listen on for connections.

cpnetserver_port = *port-number*
: Specifies the port number to use for listening.

cpnetserver_blacklist = *host-specs...*
: Whitespace-separated list of hostnames/IP-addresses that are not allowed to connect. A string beginning with a period will match all hostnames on that domain. For example, the string “.censys-scanner.com” will match (reject connections from) all hosts in the domain “censys-scanner.com”. An IP address may include a mask length, such as “45.146.164.179/24”, to match a range of addresses. This property is typically only useful for servers that listen on public interfaces.

cpnetserver_temp = *tmp-drv*
: Specifies the CP/M drive letter to designate as the temporary drive. Default is “P”. The server does not use the temporary drive, but CP/M applications may. For example, MAIL.COM uses the temporary drive as the location for mail message files.

cpnetserver_sid = *node-id*

Specifies this server's node ID on CP/NET. All CP/NET requests sent to this server must have their destination node ID (DID field) set to this value. Value is interpreted as a hexadecimal string, and must be in the range 00-FE.

cpnetserver_max = *num-clients*
Specifies the maximum number of clients that may have active connections at the same time. Values may range from 1 to 16. Clients are not required to login (login and logoff is not used). However, the first request from a client will "register" itself on the server, provided the maximum has not been exceeded.

cpnetserver_root_dir = *path*
Specifies the top-level (root) directory to be exported to CP/NET. Subdirectories named "a" through "p" are assumed, but not created automatically. Default will be "**~/HostFileBdos**".

cpnetserver_nfile = *num*
Specify the number of open files to allow. Minimum/default is 32. Some CP/M utilities, such as PIP (for 2.2), do not close files that are only read and this can result in "Too many open files" errors, for example during a long PIP transfer from a networked drive.

cpnetserver_nosys
Disable the CP/M SYS attribute, so files will not be hidden on Windows.

cpnetserver_drive_*X* **=** *path*
Where "*X*" is one of "a" through "p". Specify the path to use instead of "root_dir/X" for the exported CP/M drive.

cpnetserver_lst*X* **=** *lst-spec*
Where "*X*" is a hexadecimal LST: device number, 0-F. Specify the implementation of the LST: device. The supported strings are:
"*>file*" - send all printer out to *file*. Note, this file will contain all output sent during the life of the server.
"**Diablo630Stream** [*options*]" - Use a basic emulation of the Diablo 630 daisy-wheel printer, producing Postscript output. See section on the Diablo 630 implementation for options.
At this time, no other printer handlers exist.

cpnetserver_debug
Enables debug mode. Currently dumps messages received and sent (in hex).

Starting the Server

Typical setup will include creating a configuration file that specifies all the necessary parameters. Then invoke the JAR file while specifying the configuration file. If stderr has not been redirected (using the log property/parameter) then the terminal will have messages displayed.

Typical startup command:

```
java -jar CpnetSocketServer.jar conf=file
```

Other techniques may be used to start the server in the background or set it up to start automatically whenever the system is booted.

Diablo 630 Implementation

The Diablo 630 printer emulation is provided for convenience in serving printers to CP/NET clients. It is mostly a complete emulation. Magic Wand and WordStar use the printer's micro-spacing, and do not depend on the printer's "word processing" functions like print bold, or center a line.

Output will be turned into Postscript and sent to a file. Upon receipt of the CP/NET "end list" character (0xff), the output file will be closed and disposed of as configured (may be deleted depending on the configuration). Configuration properties may reside in the same file used for CpnetSocketServer. Similar to CpnetSocketServer, properties may also be used as commandline parameters by removing the prefix. The commandline in this case is the "[options]" part of the CpnetSocketServer LST: property.

See https://github.com/durgadas311/diablo630/blob/master/doc/VirtualDiablo630.pdf for properties and other information.

NOTE: currently, font style is not parsed. The font will always be "PLAIN".

Network Boot

Booting off the network is supported. Network boot images are stored in a single directory, specified by a property in the configuration file. By default, the directory "~/NetBoot" is used. Properties are:

netboot_dir = *path*
: Use *path* as the directory to search for network boot images. Default is **~/NetBoot**.

netboot_default = *file*
: Use *file* as the default boot image. If not specified, the file **defboot.sys** will be the default.

netboot_org0 = *file*
: Use *file* as the special binary file to be executed before starting the booted OS image. This is for systems like the Heathkit H8/H89 where RAM at 0000h needs to be enabled before the OS can start. Default is org0boot.bin. The binary is loaded into 3000h on the target machine, and executed. Bytes 3 and 4 contain the true entry point of the booted OS, so that the ORG0 program can jump to the OS start. The value "**none**" will disable any ORG0 execution.

netboot_org0_adr = *address*
: Use *address* as the load/start address for the ORG0 binary. Default is 0x3000.

Network Boot Protocols

The protocol used is as follows. Refer to CP/NET documentation for packet fields.

1. For boot stand-alone OS (or program) images, the client sends a boot request message, optionally containing a string representing the desired image. FMT=0B0H, FNC=1. String is NUL-terminated. The image file format is as described for a “SYS” image below.
2. If server refuses (or has an error), response is FMT=0B1H, FNC=0
3. If server accepts, and will send boot code, response is FMT=0B1H, FNC=1, data is a ‘$’-terminated signon string, typically containing a load map (similar to CPNETLDR output).
4. Assuming no error, client sends an ACK message to indicate it is ready for the next packet. FMT=0B0H, FNC=0
5. The server then sends a series of “set DMA” and “load record” messages, each of which must be ACK’ed by the client. Data records are always 128 bytes.
    1. FMT=0B1H, FNC=2, set DMA address to value in DAT (16 bit little-endian).
    2. FMT=0B1H, FNC=3, load 128 bytes of data into DMA, increment DMA by 128.
    3. FMT=0B1H, FNC=4, boot is finished, jump to address in DAT (16 bit little-endian).
6. If the client sends an invalid ACK packet (not FNC=0), the boot will be terminated.

An alternate boot method is the over-the-network equivalent of CPNETLDR, where an existing BIOS and BDOS are preserved and only the SNIOS and NDOS images are added.

1. For CP/NET loader booting, the client sends a message that contains the addresses of the BIOS and BDOS, optional CP/NET mappings, and an optional string to override the default image. FMT=0B0H, FNC=2, the first byte of data is the BIOS page, the second is the BDOS page, then follows zero-or-more mappings which must be pairs of bytes with the first byte’s high bit set (0x80). Then follows a string of zero-or-more characters, which is prefixed with the string length and terminated by a NUL character (an empty string consists of 00 00).

    Mappings have the high bit of the first byte set, with the remote resource 0-15 in the low four bits. The second byte is the local resource to which the map applies, 0-15 for drives A:-P: and 17 for LST:.

Network Boot “SYS” Image Format

Files that may be booted by CpnetSocketServer are in an extended CPM3.SYS file format. This also means that an actual CPM3.SYS file may be booted over the network (however, this file has no network support, so the booted OS is operating locally until CP/NET is started). This can, however, be a convenient recovery mechanism if the CP/M 3 partition CPM3.SYS file has been lost or broken.

The general format is as follows.

- The first 128 byte record contains header information:
    - 0: memtop: Top page of memory, “00” means 64K.

    - 1: comlen: Common memory code size, in pages.
    - 2: bnktop: Top page of banked memory, “C0” if bank boundary is 0xC000.
    - 3: bnklen: Banked memory code size, in pages.
    - 4,5: entry: Program start address, in real memory, little-endian.
    - 6,7: cfgtbl: (CP/NET images only) Network Config Table address, in real memory, little-endian.
    - 16: Start of the Copyright message field. If this byte contains ‘C’ then the ORG0 code will be executed.
- The second 128 byte record contains the ‘$’-terminated signon message. If there is no message, the first byte of the record will be ‘$’.
- Subsequent 128 byte records represent the code image. Common memory code is first, followed by banked memory code. Note that these records are in reverse order, they are to be loaded from the top of each memory region downwards.

There is a JAVA tool available that will combine SPR files into a boot image. This is similar to what GENCPM does, however it does not perform any of the special tasks related to configuring CP/M 3.

Note that the boot image need not be an actual OS. Stand-alone programs can be executed off the network without ever booting the machine. This can also provide a mechanism to obtain core dumps of a failing system, provided it is functional enough to boot from the network.

Network Boot “SYS” File Selection

The following search order is used to select the file to boot from. In the following, *default* refers to the value of the **netboot_default** property (which defaults to “cpnos.sys”). *XX* refers to the client node ID requesting the boot.

1. If a *tag* string was provided with the boot command:
    1. **cid***XXtag*
    2. **cid***XXtag***.sys**
    3. *tag*
    4. *tag***.sys**
2. If no *tag* is provided:
    1. **cid***XX***.sys**
    2. *default*

Network Loader Image Format

For CPNETLDR style booting, two images files are used: SNIOS.SPR and NDOS.SPR. These are the same files as would be used for CP/NET 1.2 loading via CPNETLDR.COM. This style of network boot is the functional equivalent of CPNETLDR, with additional functionality of pre-mapping some devices and being able to choose the version of the image files.

Network Loader File Selection

The following search order is used to select the file to boot from. In the following, *XX* refers to the client node ID requesting the boot, and *base* refers to the basename of the file being loaded, either “snios” or “ndos”.

1. If a *tag* string was provided with the boot command:
    1. *baseXXtag***.spr**
    2. *basetag***.spr**
2. If no *tag* is provided:
    1. *baseXX***.spr**
    2. *base***.spr**

Note that “snios” and “ndos” need not match the same variation. For example, there may be client-specific SNIOS files but only one NDOS file used by all.